

**МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ «ПОЛТАВСЬКА ПОЛІТЕХНІКА
ІМЕНІ ЮРІЯ КОНДРАТЮКА»**

**Навчально науковий інститут інформаційних технологій і робототехніки
Кафедра комп'ютерних та інформаційних технологій і систем**

Лабораторна робота № 3
з навчальної дисципліни
«Технології на платформі .NET»

Виконала:
студентка 403-ТН
Кіріяк Ірина Вікторівна

Перевірив:
Тютюнникт *Петро*
Богданович

Полтава 2025

Лабораторна робота №3

Робота із базами даних в .NET. Використання бібліотеки Entity Framework.

Передумова

Виконайте лабораторну роботу №2. Створіть нову гілку у раніше створеному гіт репозиторії NUPP_NET_2025_{Номер Групи}_ТК_{Прізвище}_Lab із назвою lab3, яка міститиме код із другої лабораторної роботи.

Для задачі лабораторної роботи, необхідно буде створити пул реквест із гілки lab3 у master, або якщо на момент задачі пул реквест із гілки lab2 у master не закритий - із гілки lab3 у lab2.

Завдання

Створити проєкт {games}.Infrastructure. У новоствореному проєкті створити папку Models. У створеній папці додати модель(набір класів), яка відповідає створеній у першій лабораторній роботі моделі, та містить всередині себе властивості. Наприклад, якщо існував у першій лабораторній клас Bus, у папці Models необхідно створити клас BusModel.

Отримана модель має обов'язково мати зв'язки один-до-одного та один-до-багатьох. За додавання зв'язків багато-до-багатьох можна отримати додаткові бали. Для наслідування більш пріоритетно застосовувати підхід Таблиця на тип (Table-per-Type).

```
PS D:\Unic\NET\NUPP_NET_2025_403_TK_Kirliak_Lab> git checkout lab2
Already on 'lab2'
PS D:\Unic\NET\NUPP_NET_2025_403_TK_Kirliak_Lab> git pull origin lab2
From https://github.com/Irakkkk/NUPP_NET_2025_403_TK_Kirliak_Lab
* branch      lab2      -> FETCH_HEAD
Updating 1757f32..20fd51d
Fast-forward
 lab2net.pdf | Bin 0 -> 273167 bytes
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 lab2net.pdf
PS D:\Unic\NET\NUPP_NET_2025_403_TK_Kirliak_Lab> git checkout -b lab3
Switched to a new branch 'lab3'
PS D:\Unic\NET\NUPP_NET_2025_403_TK_Kirliak_Lab> dotnet new classlib -n Games.Infrastructure
The template "Class Library" was created successfully.

Processing post-creation actions...
Restoring D:\Unic\NET\NUPP_NET_2025_403_TK_Kirliak_Lab\Games.Infrastructure\Games.Infrastructure.csproj:
Restore succeeded.

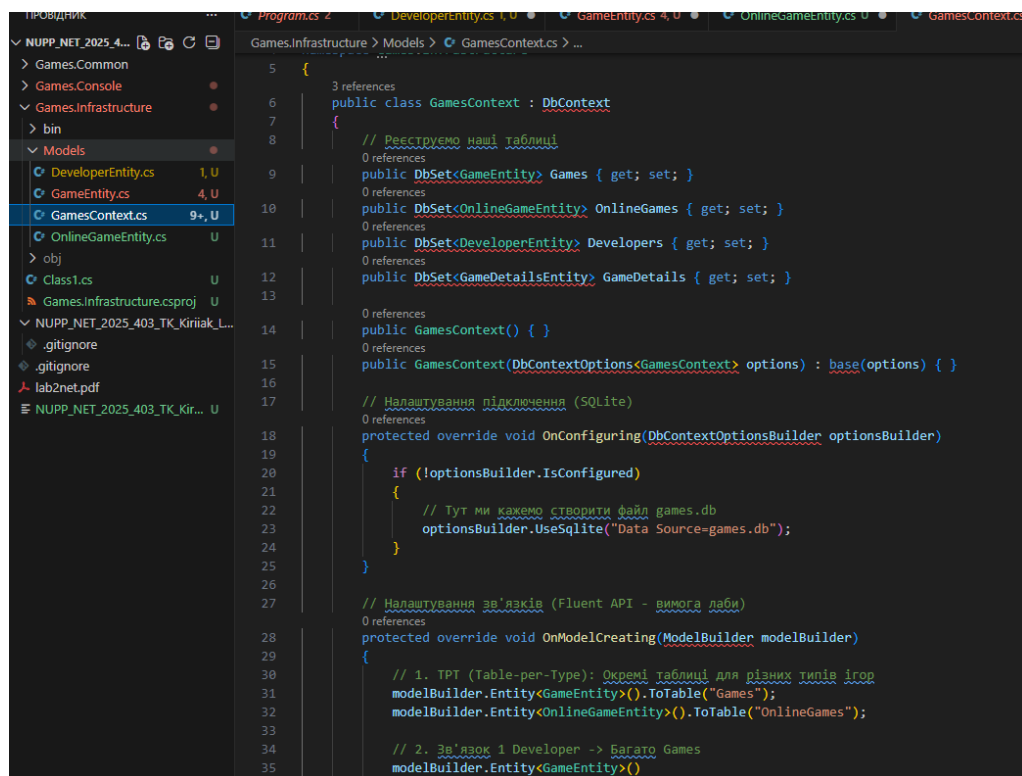
PS D:\Unic\NET\NUPP_NET_2025_403_TK_Kirliak_Lab> dotnet sln add Games.Infrastructure\Games.Infrastructure.csproj
Project `Games.Infrastructure\Games.Infrastructure.csproj` added to the solution.
PS D:\Unic\NET\NUPP_NET_2025_403_TK_Kirliak_Lab> dotnet add Games.Infrastructure\Games.Infrastructure.csproj ref
```

Створити клас {games}Context, який наслідуватиметься від класу DbContext та опише схему бази даних для обраної тематики застосовуючи Entity Framework та анотації або Fluent API на вибір, Fluent API більш пріоритетно.

Створити міграцію використовуючи Entity Framework. Створену міграцію застосувати до реляційної бази даних. Можна використовувати будь-яку реляційну СУБД, як MS SQL, PostgreSQL, MySQL, тощо. Також слід використати SQLite, коли немає можливості розробляти та здавати Лабораторну роботу на одній системі. При використанні SQLite файл бази даних необхідно додати до гіт репозиторія.

Реалізувати шаблон проєктування Репозиторій, {Назва тематики}Context який буде використовувати клас {Gamres}Context для досьупу до даних із БД:

```
public interface IRepository<T> where T : class
{
    Task<T> GetByIdAsync(int id);
    Task<IEnumerable<T>> GetAllAsync();
    Task AddAsync(T entity);
    Task Update(T entity);
    Task Delete(T entity);
}
```



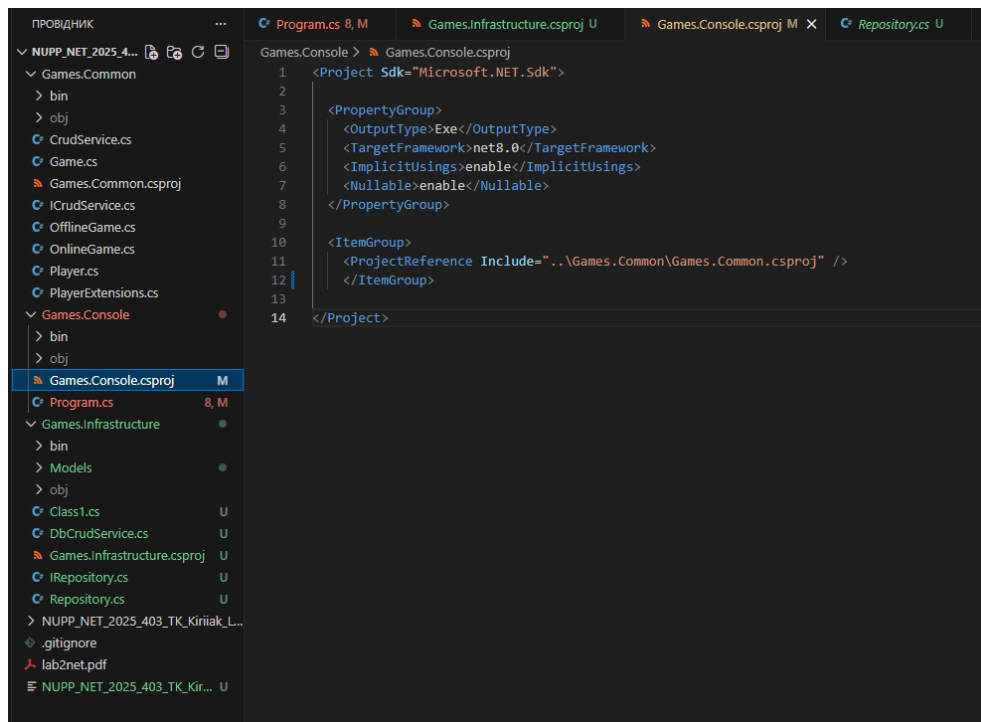
Оновити асинхрону версію дженерік CRUD сервісу, щоб він використовував репозиторій для доступу до даних:

```

public interface ICrudServiceAsync<T>
{
    public Task<bool> CreateAsync(T element);
    public Task<T> ReadAsync(Guid id);
    public Task<IEnumerable<T>> ReadAllAsync();
    public Task<IEnumerable<T>> ReadAllAsync(int page, int amount);
    public Task<bool> UpdateAsync(T element);
    public Task<bool> RemoveAsync(T element);
    public Task<bool> SaveAsync();
}

```

Модифікуйте консольний застосунок, щоб він використовував оновлену версію CRUD сервісу та дані із бази даних.



```

added: Cyberpunk 2077
added: Dota 2
added: Minecraft
added: Apex Legends
added: The Witcher 3

--- Зчитуємо всі ігри з Бази Даних ---
[ONLINE] Cyberpunk 2077 (RPG) | 2020 | 45 гравців
[ONLINE] Dota 2 (МОБА) | 2013 | 88 гравців
[ONLINE] Minecraft (Survival) | 2011 | 100 гравців
[ONLINE] Apex Legends (Battle Royale) | 2019 | 60 гравців
[ONLINE] The Witcher 3 (RPG) | 2015 | 1 гравець

```

Бази даних та СУБД

1. Що таке реляційні бази даних? Що таке СУБД?

- **Реляційна база даних** — це спосіб зберігання даних у вигляді таблиць, які логічно пов'язані між собою (через спільні поля).
- **СУБД (Система Управління Базами Даних)** — це програмне забезпечення, яке дозволяє створювати бази даних, керувати ними та надавати до них доступ (це "двигун", який виконує запити).

2. Які СУБД ви знаєте?

- SQLite (використовували в лабі), PostgreSQL, Microsoft SQL Server, MySQL, Oracle Database.

3. Що позначає термін таблиця у реляційній БД?

- **Таблиця** — це структура даних, що складається з **рядків** (записів/екземплярів сутності) та **стовпців** (атрибутів/властивостей).

4. Які існують зв'язки у реляційній БД?

- **Один-до-одного (1:1)**: Одному запису в таблиці А відповідає один запис в таблиці Б.
- **Один-до-багатьох (1:N)**: Одному запису в таблиці А відповідає багато записів в таблиці Б.
- **Багато-до-багатьох (N:N)**: Багатьом записам в таблиці А відповідає багато записів в таблиці Б (реалізується через третю проміжну таблицю).

5. Що таке ERD-діаграма?

- **ERD (Entity Relationship Diagram)** — це графічна схема, яка показує сутності (таблиці) та зв'язки між ними. Вона потрібна для проектування архітектури бази даних перед написанням коду. Її можна створити в Draw.io, DBeaver або згенерувати автоматично в Visual Studio/Rider.

Entity Framework Core (EF Core)

6. Що таке DbContext?

- **DbContext** — це головний клас в Entity Framework, який виступає "мостом" між вашим кодом (C#) та базою даних. Він керує підключенням, відстежує зміни в об'єктах та перетворює C#-код на SQL-запити.

7. Зв'язки в контексті EF Core:

- **1-до-1:** Сутність має посилання на іншу сутність (наприклад, Game має властивість GameDetails).
- **1-до-багатьох:** Сутність має колекцію інших сутностей (наприклад, Developer має List<Game>).
- **Багато-до-багатьох:** Дві сутності мають колекції одна одної (наприклад, Student має List<Course>, а Course має List<Student>).

8. Як реалізувати зв'язок один-до-одного за допомогою Fluent API?

Він налаштовується в методі OnModelCreating класу DbContext:

C#

```
modelBuilder.Entity<GameEntity>()
    .HasOne(g => g.Details)    // Гра має одні Деталі
    .WithOne(d => d.Game)      // Деталі належать одній Грі
    .HasForeignKey<GameDetailsEntity>(d => d.GameId); // Зовнішній ключ
```

9. Приклад реалізації зв'язку багато-до-багатьох у EF Core.

У сучасних версіях EF Core це робиться дуже просто (автоматично створюється проміжна таблиця):

C#

```
// У класі Student
public List<Course> Courses { get; set; }
// У класі Course
public List<Student> Students { get; set; }
```

// Fluent API (необов'язково, якщо назви стандартні, але для точності):

```
modelBuilder.Entity<Student>()
    .HasMany(s => s.Courses)
    .WithMany(c => c.Students);
```

10. Data Annotations vs Fluent API.

- **Data Annotations:** Це атрибути прямо над властивостями класу (наприклад, [Key], [Required]). Це просто, але змішує логіку БД з класом.
- **Fluent API:** Це налаштування в окремому методі OnModelCreating. Це більш гнучко, потужно (дозволяє налаштувати те, що неможливо атрибутами) і залишає класи моделей "чистими". **Fluent API має вищий пріоритет.**

11. Як створюється та застосовується міграція?

- **Створення:** Команда `dotnet ef migrations add НазваМіграції` (аналізує зміни в коді і створює файл інструкцій).
- **Застосування:** Команда `dotnet ef database update` (виконує інструкції на реальній базі даних, створюючи таблиці).

Архітектура

12. Мета проєкту **.Infrastructure**?

- Його мета — відділити логіку доступу до даних (робота з БД, файлами, зовнішніми API) від основної бізнес-логіки програми. Він відповідає на питання "ЯК зберігати дані", а не "ЩО з ними робити".

13. Різниця між доменною моделлю та моделлю БД.

- **Доменна модель (з лаби 2):** Описує бізнес-сутність, має поведінку (методи) і не залежить від бази даних.
- **Моделю БД (Entity з лаби 3):** Це "зліпок" таблиці. Вона містить тільки дані (властивості) та службові поля (ID, зовнішні ключі) для коректного збереження в базу.

14. Яку проблему вирішує шаблон Репозиторій?

- Він створює шар абстракції між програмою та базою даних. Це дозволяє легко змінити базу даних (наприклад, з MySQL на SQLite) або підмінити базу даних на тестові дані (Mock) без переписування всієї програми.

15. Принципова різниця між реляційними та нереляційними БД.

- **Реляційні (SQL):** Мають жорстку структуру (схему), дані зберігаються в таблицях, гарантують цілісність зв'язків (ACID).
- **Нереляційні (NoSQL):** Гнучкі, не вимагають фіксованої схеми. Дані можуть зберігатися як документи (JSON), пари ключ-значення або графи. Вони легше масштабуються, але мають слабші гарантії зв'язності даних.